

XPath Locators in Playwright

What is XPath?

XPath (XML Path) is used to **find elements on a web page using the HTML structure.**

In Playwright, you can use XPath like this:

```
await page.locator('//xpath_here');
```

Absolute vs Relative XPath

Absolute XPath	Relative XPath
Starts with /	Starts with //
Uses full path from root (html)	Can start from anywhere
Does not depend on attributes	Mostly uses attributes like id, class
Traverses entire DOM step-by-step	Directly targets elements

Application Used:

<https://demowebshop.tricentis.com/>

Common XPath Selectors

1. Absolute XPath (Full XPath)

What it does:

Selects element using complete path from root.

Format:

```
//html/body/div/div[2]/div[1]
```

Example:

```
//html[1]/body[1]/div[4]/div[1]/div[1]/div[1]/a[1]/img[1]
```

Example Element:

Logo

When to use:

- Note preferred (or Rarely used)

Drawback:

- Breaks easily if page structure changes

2. Relative XPath (Partial XPath)**What it does:**

Selects element from anywhere using //

Format:

```
//tagname[@attribute='value']
```

Example:

```
//img[@alt='Tricentis Demo Web Shop']
```

Example Element:

Logo

When to use:

- Most commonly used
- Short and flexible

3. XPath with Single Attribute**What it does:**

Selects element using one attribute

Format:

```
//tagname[@attribute='value']
```

Example:

```
//input[@id='small-searchterms']
```

Example Element:

Search box

4. XPath with Multiple Attributes

What it does:

Selects element using multiple attributes

Format:

```
//tagname[@attr1='value1'][@attr2='value2']
```

Example:

```
//input[@value='Search'][@type='submit']
```

Example Element:

Search button

5. XPath with 'and' Operator

What it does:

All conditions must be true

Format:

```
//tagname[@attr1='value1' and @attr2='value2']
```

Example:

```
//input[@value='Search' and @type='submit']
```

Example Element:

Search button

6. XPath with 'or' Operator

What it does:

Any one condition should be true

Format:

```
//tagname[@attr1='value1' or @attr2='value2']
```

Example:

```
//input[@value='Search' or @type='submit']
```

Example Element:

Search button

7. contains()**What it does:**

Matches partial attribute value

Format:

```
//*[contains(@attribute,'value')]
```

Example:

```
//h2//a[contains(@href,'computer')]
```

Example Element:

Products containing "computer"

When to use:

- When attribute values are dynamic

8. starts-with()**What it does:**

Matches starting part of attribute value

Format:

```
//*[starts-with(@attribute,'value')]
```

Example:

```
//h2//a[starts-with(@href,'/build')]
```

Example Element:

Products starting with "build"

9. text()

What it does:

Matches element using visible text

Format:

```
//*[@text()='value']
```

Examples:

```
//*[@text()='Register']  
//*[@contains(text(),'Fiction')]
```

Example Element:

Register link

10. last()

What it does:

Selects last element in a group

Format:

```
//tagname[last()]
```

Example:

```
//div[@class='column follow-us']//li[last()]
```

Example Element:

Last social link in footer

11. position()

What it does:

Selects element by index (1-based)

Format:

//tagname[position()=n]

Example:

//div[@class='column follow-us']/li[position()=2]

Example Element:

Selects the 2nd option under the FOLLOW US footer menu.

Important:

- Index starts from **1**

Dynamic XPath

Sometimes element attributes change dynamically.

Example HTML:

```
<button name="start">START</button>
<button name="stop">STOP</button>
```

Problem:

- Values change dynamically (start ↔ stop)

Solutions:

XPath	Technique
//button[@name='start' or @name='stop']	using OR
//button[contains(@name,'st')]	contains
//button[starts-with(@name,'st')]	starts-with
//button[text()='STOP' or text()='START']	text

Summary

- Absolute XPath → full path (avoid)
- Relative XPath → flexible and preferred
- @attribute → select by attribute
- and / or → combine conditions
- contains() → partial match
- starts-with() → prefix match
- text() → match visible text
- last() → last element
- position() → index-based selection

Tips

- Prefer **relative XPath**
- Keep XPath simple and readable
- Use contains() for dynamic values
- Avoid long and complex XPaths
- Use XPath only when CSS or Playwright locators are not sufficient